USER GUIDE MANUAL – 8-BIT MICROPROCESSOR

A Project by Computer Engineering Students

Course Title and Section: Colegio de Montalban
Instructor: Engr. Emerson Facelo
Date: May 25, 2025

Team Members:

1. Kenneth Jearl S. Gonzales
2. French John D. Del Valle
3. John Martin Corpin
4. Kenneth Aguirre
5. Christian Pedrita
6. Kyle Martin Lora
7. Ben Rhovic Lao
8. Gabriel Roque
9. Adrian Buenaobra
10. Khaye Calma
11. Israel Beriña
12. Rex Pitogo
13. Clifford Masing

Table of Contents

1. Introduction

This project implements a simplified **8-bit microprocessor emulator** on an **ESP32 microcontroller**, designed to introduce core computing concepts through practical, hands-on interaction. Inspired by the Intel 8086 architecture, the emulator models a minimal processor with a reduced instruction set and 8-bit data width.

- **Registers**: Four general-purpose registers — **A, B, C, and D** — each capable of holding 8-bit values (0–255).
- **Instruction Set** (Total: 9 instructions):
  - **Data Transfer**: MOV, XCHG
  - **Arithmetic**: ADD, SUB, NEG
  - **Logical**: AND, OR
  - **Register Modification**: INC, DEC

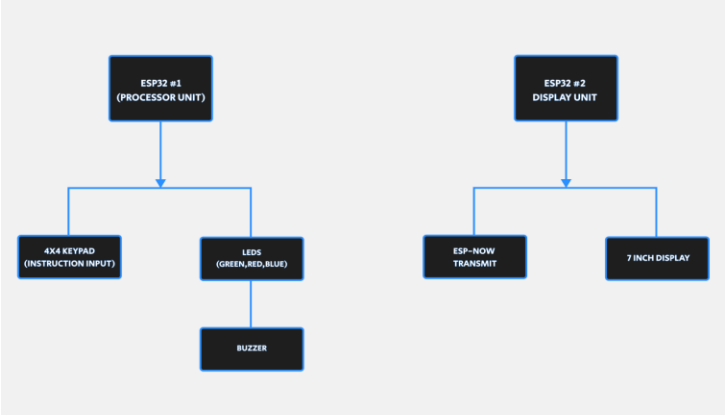
This emulator offers a clear, interactive platform for learning low-level programming and microprocessor fundamentals, including register-based data manipulation and binary logic operations. While inspired by Intel 8085 **(16-bit)**, it is intentionally constrained to **8-bit operations** and a **limited instruction set**, making it ideal for educational and demonstration purposes.

2. System Overview

Serving as both an educational tool and practical embedded systems prototype, it combines physical input through a 4×4 matrix keypad with wireless communication via ESP-NOW protocol to drive a remote TFT display, creating a complete workflow from instruction entry to visual output. The design emphasizes feedback buzzer tones that reinforce system states and operation outcomes.

Components	Functions
ESP32 #1 – Processor Unit	• Acts as the main processing unit (the "emulated microprocessor") • Handles all instruction processing, register operations (A, B, C, D), and logical/arithmetic execution
ESP32 #2 – Display Unit	• Dedicated to fetching data from the Processor Unit • Displays the current register values, operation status, and feedback on a 7-inch TFT display • Reduces processing load on the main processor by offloading display rendering
4x4 Matrix Keypad	Allows user to input instructions and select operations
7-inch TFT Display	Displays register states and operation results in real-time
Buzzer	Provides feedback sound on specific actions (e.g., successful, errors)
3 LEDs	Show operation feedback and activity • Green = success/status OK • Red = error/invalid operation • Blue = any keypress

Power Supply	Supplies regulated power to ESP32 boards and peripherals
PCB	Integrates all components for stable and organized layout
24AWG Wires	For connecting components securely and handling sufficient current

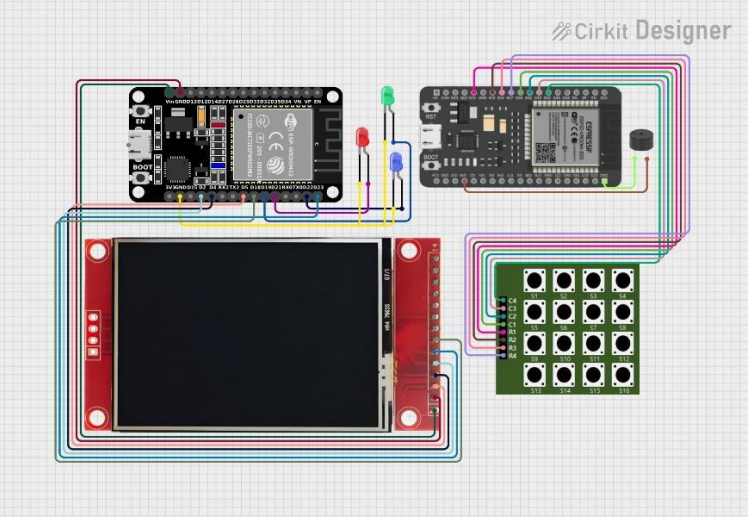


3. Features

This section outlines the technical specifications of the 8-bit microprocessor system, including its data width, instruction set, clock speed, and I/O support. The system utilizes two ESP32 microcontrollers—one dedicated to handling input from a 4×4 matrix keypad, and the other for managing output to a TFT LCD display designed to replicate the behavior of a simplified 8-bit processor through software logic.

Feature	Description
Data Width	The system is based on an 8-bit data architecture, with four general-purpose registers: **regA**, **regB**, **regC**, and **regD**. Each register can hold values ranging from **0 to 255**, mimicking the behavior of traditional 8-bit microprocessors. Arithmetic operations are masked using **& 0xFF** to enforce the 8-bit constraint.
Instruction Set	9 total instructions: MOV, XCHG, ADD, SUB, NEG, AND, OR, INC, DEC
Registers	4 General Purpose Registers: A, B, C, D (each 8- bit)
Clock Speed	Based on ESP32: up to 240 MHz (programmable for timing simulation)
Microcontroller(s)	2x ESP32 (1 for processing, 1 for display communication using ESP-NOW)
Input Interface	4x4 Matrix Keypad (16 keys for instruction entry and control)
Output Display	7-inch TFT LCD (displays register states, operations, and status)
Indicators	3 LEDs: Green (Success), Red (Error), Blue (Keypress Activity)
Audio Feedback	Piezo Buzzer (for status tones or error signals)
Transmission	ESP-NOW (wireless protocol between ESP32 Processor and Display Unit)

4. Hardware Setup



5. Instruction Set Architecture (ISA)

INSTRUCTION	OPERATION	DESCRIPTION
MOV	Move	Input value into a register
ADD	Add	Adds two registers
SUB	Subtract	Subtracts two registers
AND	Logical AND	Bitwise AND of two registers
OR	Logical OR	Bitwise OR of two registers
INC	Increment	Increase register value by 1
DEC	Decrement	Decrease register value by 1
NEG	Negate	Two's complement a register
XCHG	Exchange	Swap values of two registers

6. How to Use

This device lets you manipulate 4 virtual registers: A, B, C, D using a 4x4 keypad. You can:

- Move values
- Add/Subtract/Negate
- Perform bitwise operations
- Increment/Decrement
- Swap values

Understanding the Keypad Commands

1	**2**	**3**	**A**
4	**5**	**6**	**B**
7	**8**	**9**	**C**
RES	**0**	**EXE**	**D**

Step-by-Step Instructions

Set Register to a Value	1. Press 1 (MOV). 2. Press target register (A, B, C, or D). 3. Press numeric keys (0–9) to input a value (up to 3 digits). 4. Press EXE to execute.
Move Value from One Register to Another	1. Press 1 (MOV). 2. Press target register. 3. Press source register. 4. Press EXE.
Add / Subtract	1. Press 2 (ADD) or 3 (SUB) 2. Press target register. 3. Press value (number keys) or another register (A–D). 4. Press EXE.
Bitwise AND / OR	1. Press 4 (AND) or 5 (OR). 2. Press target register. 3. Press source register. 4. Press EXE.
Swap Registers (XCHG)	1. Press 9. 2. Press register 1. 3. Press register 2. 4. Press EXE.
INC / DEC / NEG	1. Press 6, 7, or 8. 2. Press register to modify. 3. Press EXE.
Reset All Registers	Press RES (no EXE needed).

LED and Buzzer Feedback

Indicator	Meaning
Blue LED	Keypress registered
Green LED	Operation success
Red LED	Invalid input / overflow
Buzzer	Feedback confirmation

7. Sample Applications

1. Intel 8051 Microcontroller Emulator

This project involves the development of an emulator for the Intel 8051 microcontroller, implemented using modern web technologies such as Brython and React.
The emulator replicates the functionality of the 8051 architecture, allowing users to simulate and interact with the microcontroller's operations within a browser environment.
While not directly utilizing the ESP32, the project's emphasis on emulating classic microcontroller behavior parallels the objectives of creating an 8086 emulator on contemporary hardware platforms.

Reference:

GitHub Repository: estarq/i8051emu

2. Sci-Calc: An Open-Source Multifunctional Scientific Calculator Using ESP32

The Sci-Calc project presents a multifunctional device that integrates a scientific calculator, a handheld gaming console, an ESP32 development board, and a customizable mechanical numpad into a single compact unit.
Powered by the ESP32 microcontroller, the device showcases the versatility of the platform in handling complex computational tasks and user interfaces. The project's open-source nature and emphasis on scientific computation align closely with the goals of developing an 8-bit scientific calculator emulator.

Reference:

GitHub Repository: shaoxiongduan/sci-calc

3. RunCPM: A CP/M Emulator for ESP32

RunCPM is a portable emulator that allows the execution of CP/M 2.2 programs on modern hardware platforms, including the ESP32.
By emulating the Z80 processor, RunCPM enables the operation of vintage software such as WordStar and MBASIC on contemporary microcontrollers. This project demonstrates the feasibility of replicating classic computing environments on modern hardware, mirroring the objectives of emulating the Intel 8086 architecture for educational and practical applications

Reference:

GitHub Repository: MockbaTheBorg/RunCPM

8. Troubleshooting Guide

1. No Power or Device Not Powering On

Power Verification: Confirm that the power supply delivers the appropriate voltage (typically 3.3V or 5V) to all components using a multimeter.

USB Port Inspection: Ensure that the USB connection is correctly soldered and that the onboard voltage regulator is functional.

Continuity Testing: Perform continuity checks on the power and ground lines to verify that there are no open circuits or broken traces on the PCB.

2. TFT Display Not Functioning

GPIO Pin Mapping: Ensure the correct connections between the ESP32 GPIO pins and the TFT display pins according to the project's wiring diagram.

Soldering Quality: Inspect for cold solder joints or solder bridges, especially on the display header pins. Re-solder if necessary.

Code Validation: Upload a basic test sketch to verify display functionality and eliminate firmware-related issues.

3. Keypad Not Responding or Incorrect Inputs
Connection Verification: Confirm that the keypad matrix is correctly routed to the ESP32 and that no connections are loose or reversed.

Mechanical Integrity: Check the physical condition of the push buttons to ensure they are not damaged or misaligned.

Firmware Mapping: Verify that the software configuration matches the keypad's wiring layout on the PCB.

4. ESP32 Not Detected by Development Environment
USB Communication Lines: Test the USB data lines for continuity and ensure proper connection between the USB port and the ESP32 chip or USB-to- Serial converter.

Boot Configuration: Ensure that the BOOT and EN (Enable) buttons are working correctly and that the necessary pull-up or pull-down resistors are in place for upload mode.

Short Circuit Check: Investigate for potential shorts between power and ground lines which may prevent the ESP32 from initializing.

5. Non-Functioning LEDs
Orientation Check: Verify that each LED is installed with the correct polarity.
Current Limiting Resistors: Ensure that appropriate resistors are installed in series with the LEDs to prevent overcurrent.
GPIO Control: Confirm that the firmware correctly addresses the GPIOs connected to the LEDs.

6. Short Circuits After Assembly
Visual Inspection: Examine the PCB closely for solder bridges, particularly in densely populated areas such as the microcontroller headers and keypad interface.

2. Software Maintenance
a. Code Robustness
- Use debouncing algorithms for keypad input.

Continuity Measurement: Use a multimeter to detect any unintended connections between adjacent pins or between VCC and GND.

7. Instruction Execution Errors
Opcode Decoder Logic: Review the logical implementation of the instruction decoder to ensure correct interpretation of fetched opcodes.

Memory Interface: Confirm that the memory modules are correctly connected and accessible by the microcontroller without address conflicts or data corruption.

Maintenance and Safety

1. Hardware Maintenance
a. Regular Inspection
- Inspect solder joints for cracks or dry joints.
- Ensure no burnt smell, discoloration, or overheating on the microprocessor or ESP32.

b. Component Care
- LEDs: Ensure current-limiting resistors are used to prevent burnout.
- Buzzer: Confirm proper voltage supply to avoid overheating.
- TFT Display: Keep dust-free; avoid direct pressure on the screen.
- Matrix Keypad: Clean regularly; avoid fluid or debris ingress.
- ESP32: Avoid ESD (Electrostatic Discharge) by handling with care.

c. Power Supply
- Use a regulated power source.
- Add protection diodes and capacitors to filter voltage spikes and prevent reverse polarity.

- Log or display error states on the TFT or via LED indicators.

b. Firmware Updates
   - Use OTA (Over-The-Air) updates cautiously if enabled.
   - Maintain version control and rollback ability.

3. Electrical and Operational Safety

a. Static Discharge Protection
- Use anti-static mats/wrist straps during assembly or maintenance.

b. Isolation
   - Isolate ESP32's USB port from high-power circuits.
   - Use optocouplers or level shifters if interfacing with higher-voltage components.

c. Overcurrent/Overvoltage Protection
   - Fuse or polyfuse at the power input
   - Zener diodes or TVS diodes for ESD and spike protection

4. Physical Handling
- Handle the microprocessor by its edges to avoid touching pins or the surface.
- Avoid bending or damaging IC pins—use IC pullers or insertion tools if needed.

5. Cleaning
- Clean the board periodically with isopropyl alcohol and a soft brush.
- Avoid using water or household cleaners on electronic boards.

9. Glossary

8-bit – A data size in which values range from 0 to 255 (2^8 possible values). It defines how much information a register or data bus can hold.
Add (ADD) – An arithmetic instruction that adds two numbers and stores the result in a register.
AND – A logical bitwise operation where only the bits that are 1 in both operands remain 1.
Buzzer – An electronic component that makes a sound to provide feedback (like an alert for errors or successful operations).
Clock Speed – The speed at which a processor executes instructions, measured in MHz. The ESP32 can go up to 240 MHz. **Data Transfer (MOV, XCHG)** – Instructions used to move or swap data between registers.
DEC (Decrement) – An instruction that decreases the value of a register by 1.
ESP32 – A powerful microcontroller with built-in Wi-Fi and Bluetooth, used here to emulate the processor and control the display.
ESP-NOW – A wireless protocol used by ESP32s for fast, low- power communication between devices without Wi-Fi setup. **EXE (Execute)** – A command used to run or confirm an instruction after selecting it using the keypad.
General Purpose Register (A, B, C, D) – A small storage area in the processor used to store data or intermediate values during computation.
INC (Increment) – An instruction that increases the value of a register by 1.
Instruction Set – The collection of all available commands (e.g., MOV, ADD, SUB) that the microprocessor can execute.
LED (Light Emitting Diode) – A small light used to indicate status: success (green), error (red), or activity (blue).
Logical Operation – Operations like AND or OR that compare binary values bit by bit.

Matrix Keypad – A 4×4 button interface used by the user to input commands and data.
NEG (Negate) – An instruction that flips the bits of a value (i.e., two's complement).
OR – A logical operation where bits are 1 if either of the corresponding bits in the operands is 1.
PCB (Printed Circuit Board) – The board that holds and connects all electronic components together.
Piezo Buzzer – A sound device that beeps when actions are performed, giving audio feedback.
Register – A storage location inside the processor used to hold data temporarily.
SUB (Subtract) – An arithmetic instruction that subtracts one value from another.
TFT Display – A screen that shows information such as register values and execution feedback.
XCHG (Exchange) – An instruction that swaps the values of two registers.
0xFF – A hexadecimal value representing the maximum 8-bit number (255 in decimal). Used to ensure values stay within 8-bit limits.

10. Appendix

A. Instruction Opcode Table.

Instruction	Opcode	Operands	Description
MOV	0x01	reg, val / reg, reg	Move value or register data
ADD	0x02	reg, val / reg, reg	Add value or register to target
SUB	**0x03**	**reg, val / reg, reg**	**Subtract value or register from target**
AND	0x04	reg, reg	Bitwise AND between registers
OR	0x05	reg, reg	Bitwise OR between registers
INC	0x06	reg	Increment value of register
DEC	0x07	reg	Decrement value of register
NEG	0x08	reg	Negate (2's complement) of register
XCHG	0x09	reg1, reg2	Exchange values of two registers

Note: All instructions are masked using & 0xFF to maintain 8-bit behavior.

B. Register Address Table

Register	Address/Label	Size
A	regA	8-bit
B	regB	8-bit
C	regC	8-bit
D	regD	8-bit

C. System Wiring Schematic (Simplified) Key Interconnections

- **Keypad to ESP32 Processor Unit**: Connected via GPIO pins (columns and rows defined in firmware).
- **ESP32 Processor → Display Unit**: Communicates via ESP-NOW.
- **TFT Display**: SPI-connected to ESP32 Display Unit.
- **Buzzer**: Driven by GPIO output (with series resistor).
- **LEDs**: Connected to GPIO pins with current-limiting resistors (Green, Red, Blue).

11. References

El-Medany, W., Al-Omary, A., & Al-Husainy, M. (2013). Design and Implementation of a Microprocessor System for Educational Purposes. Procedia Computer Science, 19, 297–304. https://doi.org/10.1016/j.procs.2013.06.041

This paper discusses the design of a microprocessor-based system for education, which aligns with the project's goal of creating a learning tool for understanding 8-bit computing and machine instructions.

GeeksforGeeks. 8086 Instruction Set. Retrieved from https://www.geeksforgeeks.org/8086-instruction-set/

The 8086 instruction set serves as a foundational reference for designing and emulating basic assembly instructions in the proposed 8-bit processor, making it a useful resource for opcode and instruction format inspiration.

Morlan, A. (n.d.). Building an 8-bit Breadboard Computer with an FPGA. Retrieved from https://austinmorlan.com/posts/8bit_breadboard_fpga/

This blog provides insights into building an 8-bit processor on a breadboard, similar to this project's hardware-focused approach, using simple components and emphasizing educational value and clarity.

Wu, Y., & Ho, Y. (2008). Design and FPGA Implementation of a Simple 8-bit Microprocessor. IEEE Conference on Industrial Electronics and Applications. https://ieeexplore.ieee.org/abstract/document/4562743

This IEEE paper explores implementing a custom 8-bit microprocessor using FPGA, which closely mirrors the conceptual and architectural design strategies applied in this project using microcontrollers.


Inspired by Intel 8086
Architecture
0xFF/255
Tiny 8086
8-Bit
Microprocessor
Emulator
Learn how a real microprocessor works